

C++ factorial logic

1. cpp logic for finding factorials.	2
2. cpp logic for finding digits of given number.	4
3. cpp logic for separating digits from a given number.	6
4. don't use vectors.	8
5. are these related to number theory.	10



1. cpp logic for finding factorials

Here's a simple C++ logic to find the **factorial** of a number. I'll provide both **iterative** and **recursive** approaches.

✓ Iterative Approach

cpp

```
#include <iostream>
using namespace std;

int factorial(int n) {
    int result = 1;
    for(int i = 2; i <= n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    int number;
    cout << "Enter a number: ";
    cin >> number;

    if(number < 0) {
        cout << "Factorial is not defined for negative numbers.";
    } else {
        cout << "Factorial of " << number << " is " << factorial(number);
    }

    return 0;
}
```

✓ Recursive Approach

cpp

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if(n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    int number;
```

```
cout << "Enter a number: ";
cin >> number;

if(number < 0) {
    cout << "Factorial is not defined for negative numbers.";
} else {
    cout << "Factorial of " << number << " is " << factorial(number);
}

return 0;
}
```

Let me know if you need a version that handles **large numbers** (e.g. using `long long` or a big integer library).



2. cpp logic for finding digits of given number

To find the **digits of a given number** in C++, you can use either a **loop with modulo and division**, or convert the number to a **string**. Here's both approaches:

✓ 1. Using Modulo and Division (Mathematical approach)

cpp

```
#include <iostream>
using namespace std;

void printDigits(int number) {
    if (number == 0) {
        cout << 0 << endl;
        return;
    }

    if (number < 0) {
        cout << "-";
        number = -number;
    }

    // Store digits in reverse order first
    int digits[10]; // assuming max 10 digits
    int count = 0;
    while (number > 0) {
        digits[count++] = number % 10;
        number /= 10;
    }

    // Print in correct order
    for (int i = count - 1; i >= 0; --i) {
        cout << digits[i] << " ";
    }
    cout << endl;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Digits: ";
    printDigits(num);

    return 0;
}
```

✓ 2. Using String Conversion (Simpler approach)

cpp

```
#include <iostream>
#include <string>
using namespace std;

void printDigits(int number) {
    string numStr = to_string(number);

    for (char ch : numStr) {
        if (ch == '-') {
            cout << "- ";
        } else {
            cout << ch << " ";
        }
    }
    cout << endl;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Digits: ";
    printDigits(num);

    return 0;
}
```



Sample Output

For input **-5031**, output will be:

```
makefile
```

```
Digits: - 5 0 3 1
```

Let me know if you want the **sum** or **count** of digits too.



3. cpp logic for separating digits from a given number

To **separate digits** from a given number in C++, you can use a loop with **modulo (%)** and **division (/)** operators. Here's a clean and simple example of how to do this — including versions to extract digits **from left to right** (in original order) or **right to left**.

✓ C++ Logic to Separate Digits (Left to Right)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

void separateDigits(int number) {
    if (number == 0) {
        cout << 0 << endl;
        return;
    }

    if (number < 0) {
        cout << "- ";
        number = -number;
    }

    vector<int> digits;

    // Extract digits in reverse order first
    while (number > 0) {
        digits.push_back(number % 10);
        number /= 10;
    }

    // Print in correct (original) order
    for (int i = digits.size() - 1; i >= 0; --i) {
        cout << digits[i] << " ";
    }
    cout << endl;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Separated digits: ";
    separateDigits(num);

    return 0;
}
```

✓ Alternate Version: Using String Conversion

cpp

```
#include <iostream>
#include <string>
using namespace std;

void separateDigits(int number) {
    string str = to_string(number);

    for (char ch : str) {
        cout << ch << " ";
    }
    cout << endl;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Separated digits: ";
    separateDigits(num);

    return 0;
}
```

Sample Output

Input:

css

Enter a number: -4205

Output:

yaml

Separated digits: - 4 2 0 5

Let me know if you want to store the digits in an array or do additional operations like **digit sum**, **even/odd separation**, etc.



4. don't use vectors

Sure! Here's a **C++ program** that separates the digits of a given number **without using vector or string**, and prints them **from left to right** (i.e., in the original order).

This version uses **math only** — just loops, integers, division, and modulus.

✓ C++ Logic (No vector, No string)

cpp

```
#include <iostream>
#include <cmath>
using namespace std;

void separateDigits(int number) {
    if (number == 0) {
        cout << "0" << endl;
        return;
    }

    if (number < 0) {
        cout << "- ";
        number = -number;
    }

    // Count how many digits the number has
    int temp = number;
    int divisor = 1;

    // Find the highest power of 10 less than or equal to the number
    while (temp >= 10) {
        temp /= 10;
        divisor *= 10;
    }

    // Use the divisor to extract digits from left to right
    while (divisor > 0) {
        int digit = number / divisor;
        cout << digit << " ";
        number %= divisor;
        divisor /= 10;
    }

    cout << endl;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "Separated digits: ";
```



```
    separateDigits(num);  
    return 0;  
}
```

Example Output

Input:

CSS

Enter a number: -7064

Output:

yaml

Separated digits: - 7 0 6 4

This version avoids using any dynamic memory structures and works purely with integer math. Let me know if you want to also store the digits in a static array.



5. are these related to number theory

Yes, the logic you're using to **separate digits** or compute **factorials** is **related to number theory**, but only at a **basic level**.

Here's a breakdown:

✔ Digit Separation

When you separate the digits of a number using division and modulo:

- You're working with **base-10 representation** of numbers.
- This touches on **number representation** in number theory.
- `number % 10` and `number / 10` are operations based on **modular arithmetic** and **integer division** — both fundamental concepts in number theory.

So yes, this is related to **elementary number theory**, though it's more **programming-focused** than theoretical.

✔ Factorial

Factorial logic ($n! = n \times (n-1) \times \dots \times 1$) is **directly tied** to number theory:

- It's used in **combinatorics**, **modular arithmetic**, and **prime number theory** (e.g., Wilson's theorem).
- Factorials are key in **binomial coefficients**, **permutations**, and **mathematical proofs**.

So factorial computation is **definitely part of number theory**.

In Summary:

Operation	Number Theory Relevance
Digit separation	Basic (number representation, mod)
Factorial	Strong (combinatorics, arithmetic)

Would you like to go deeper into number theory topics like **primes**, **GCD/LCM**, **modular inverses**, or **Euler's theorem**?